

Especificación de Arquitectura de Microservicios

Plataforma de Aprendizaje en Línea - PBY2201 Backend I

Versión: 1.1

Fecha: 2026-05-09

Autor:

- Nicolás Cavieres
- Rainiero Morroni **Proyecto:** Plataforma de Aprendizaje en Línea - PBY2201

💡 **Nota sobre Decisiones de Desarrollo vs. Despliegue Final:** > Este documento detalla la arquitectura conceptual completa diseñada para un entorno de producción real. Sin embargo, para efectos de desarrollo temprano y evaluación académica (EFT), el proyecto se ha construido temporalmente como un **Monolito Modular** utilizando una base de datos en memoria (**H2**) para pruebas. En etapas posteriores de despliegue, la aplicación se separará físicamente en un microservicio por dominio, utilizando motores de base de datos robustos (**MySQL**) y brokers de mensajería dedicados. La demostración en vivo es realizada con MySQL.

1. Análisis de Requerimientos

1.1 Requerimientos Funcionales del Sistema

La plataforma de aprendizaje en línea requiere funcionalidades clave organizadas en los siguientes dominios:

Autenticación y Gestión de Usuarios

- Registro de nuevos usuarios (estudiantes, académicos, administradores)
- Autenticación segura mediante JWT
- Gestión de perfiles y cambio de contraseña
- Control de estados de usuario (activo, bloqueado, inactivo)
- Recuperación de contraseña mediante email

Gestión de Cursos

- Creación y publicación de cursos por académicos
- Catálogo de cursos disponibles con filtrado por categoría
- Actualización de información de cursos
- Gestión de cupos disponibles
- Eliminación de cursos (solo administradores)

Inscripciones

- Inscripción de estudiantes en cursos
- Seguimiento del estado de inscripciones (pendiente pago, activa, cancelada)
- Visualización de inscripciones por estudiante
- Control de estudiantes inscritos por curso
- Cancelación de inscripciones

Tareas

- Creación de tareas por académicos
- Asignación de tareas a cursos
- Entregas de tareas por estudiantes (con archivos)
- Visualización de entregas por tarea
- Actualización y eliminación de tareas

Evaluaciones

- Creación de evaluaciones con ponderación
- Registro de calificaciones de estudiantes
- Visualización de notas por estudiante
- Modificación de calificaciones
- Eliminación de evaluaciones

Pagos

- Procesamiento de pagos de inscripciones
- Consulta de detalles de pago
- Historial de pagos del estudiante
- Solicitud de reembolsos
- Gestión de pagos (acceso administrativo)

[!!IMPORTANT] Por seguridad y consistencia, hemos decidido que no se pueden eliminar pagos.

Notificaciones

- Notificaciones manuales por administradores/académicos
- Recordatorios automáticos de tareas pendientes
- Notificaciones de resultados de evaluaciones
- Marcado de notificaciones como leídas
- Eliminación de notificaciones

1.2 Satisfacción de Requerimientos mediante Arquitectura de Microservicios

Se propone una **arquitectura basada en microservicios independientes comunicados mediante un API Gateway (REST) y un Message Broker (eventos asíncronos)**, asignando una base de datos exclusiva a cada servicio.

Este modelo satisface los requerimientos del sistema porque:

- **Aislamiento de Fallos:** Cada microservicio es independiente. Si msEvaluacion falla, no afecta el funcionamiento de msCurso o msUsuario, garantizando continuidad de servicio.
- **Consistencia Eventual:** Los datos compartidos entre servicios se mantienen sincronizados mediante eventos asíncronos, permitiendo que cada servicio gestione su propia base de datos sin transacciones distribuidas complejas.
- **Escalabilidad Independiente:** Procesos críticos de negocio (evaluaciones, pagos) pueden escalar dinámicamente sin afectar otros servicios, optimizando recursos.
- **Alta Disponibilidad:** La arquitectura distribuida asegura que estudiantes y profesores puedan acceder al sistema sin generar cuellos de botella en componentes específicos.
- **Flexibilidad Tecnológica:** Cada microservicio puede usar la tecnología más adecuada para su dominio (bases de datos, lenguajes de programación, frameworks).

2. Diseño de Microservicios

2.1 Identificación de Microservicios

Se han identificado **7 microservicios** principales con responsabilidades claramente definidas:

Microservicio	Responsabilidad Principal	Base de Datos	Protocolos
msUsuario	Autenticación, gestión de usuarios y perfiles	PostgreSQL (usuarios)	REST (HTTP/HTTPS)
msCurso	Creación y administración de cursos	PostgreSQL (cursos)	REST (HTTP/HTTPS)
msInscripcion	Control de inscripciones de estudiantes	PostgreSQL (inscripciones)	REST (HTTP/HTTPS)
msTarea	Gestión de tareas y entregas	PostgreSQL (tareas, archivos)	REST (HTTP/HTTPS)
msEvaluacion	Calificaciones y evaluaciones	PostgreSQL (evaluaciones, notas)	REST (HTTP/HTTPS)
msPago	Procesamiento de pagos e invoices	PostgreSQL (transacciones)	REST (HTTP/HTTPS)
msNotificacion	Notificaciones asincrónicas (email, push, SMS)	MongoDB (histórico flexible), Redis (cache)	Eventos (Kafka/RabbitMQ)

2.2 Responsabilidades Detalladas por Microservicio

msUsuario (Autenticación y Gestión)

Responsabilidades:

- Registro e ingreso de usuarios
- Emisión y validación de JWT
- Gestión de perfiles y roles (ESTUDIANTE, ACADEMICO, ADMIN)
- Recuperación de contraseña
- Gestión de estados de usuario

Interacciones:

- Provee validación de token a otros servicios
- Consumido por: msCurso, msInscripcion, msTarea, msEvaluacion, msPago, msNotificacion

msCurso (Gestión de Cursos)

Responsabilidades:

- CRUD de cursos
- Gestión de cupos y disponibilidad
- Categorización de cursos
- Gestión de profesores asignados

Interacciones:

- Consultado por msInscripcion (validar curso existe y tiene cupos)
- Consultado por msTarea (validar que la tarea pertenece a un curso)
- Generador de eventos: "Curso creado", "Curso actualizado", "Cupo ocupado"

msInscripcion (Control de Inscripciones)

Responsabilidades:

- Crear inscripciones de estudiantes
- Gestionar estados: PENDIENTE_PAGO → ACTIVA → COMPLETADA/CANCELADA
- Validar inscripciones duplicadas
- Listar inscripciones por estudiante o curso

Interacciones:

- Valida usuario con msUsuario
- Valida curso con msCurso
- Se comunica con msPago para actualizar estado tras pagos
- Consumidor de eventos: "Pago completado", "Pago rechazado"
- Generador de eventos: "Inscripción creada", "Inscripción cancelada"

msTarea (Gestión de Tareas)

Responsabilidades:

- CRUD de tareas por curso
- Almacenamiento de entregas (archivos)
- Validación de plazos
- Estadísticas de entregas

Interacciones:

- Valida curso con msCurso
- Valida usuario con msUsuario
- Generador de eventos: "Tarea creada", "Entrega recibida", "Entrega atrasada"
- Se comunica con msNotificacion para recordatorios

msEvaluacion (Calificaciones)

Responsabilidades:

- CRUD de evaluaciones
- Registro de calificaciones por estudiante
- Cálculo de promedios ponderados
- Historial de notas

Interacciones:

- Valida usuario y curso
- Consumidor de eventos: "Inscripción activa"
- Generador de eventos: "Calificación registrada", "Evaluación finalizada"
- Se comunica con msNotificacion para notificar resultados

msPago (Procesamiento de Pagos)

Responsabilidades:

- Procesamiento de pagos de inscripciones
- Gestión de métodos de pago
- Solicitud de reembolsos
- Generación de comprobantes

Interacciones:

- Valida inscripción con msInscripcion
- Valida usuario con msUsuario
- Generador de eventos: "Pago aprobado", "Pago rechazado", "Reembolso procesado"
- Consumidor de eventos: "Inscripción creada"

msNotificacion (Sistema de Notificaciones - Asíncrono)

Responsabilidades:

- Consumidor de eventos de todos los servicios
- Envío de notificaciones a través de múltiples canales (email, push, SMS)
- Gestión de preferencias de notificación del usuario

- Histórico de notificaciones enviadas
- Reintentos automáticos si falla el envío

Interacciones:

- **Consumidor de eventos:** Suscrito a eventos de todos los servicios
 - `usuario.registrado` → email de bienvenida
 - `inscripcion.creada` → confirmación de inscripción
 - `pago.aprobado` → confirmación de pago
 - `tarea.entregada` → confirmación de entrega
 - `evaluacion.calificada` → notificación de calificación
 - `tarea.proxima-vencer` → recordatorio (scheduled)
- **No depende de latencia:** Procesa eventos en background sin bloquear otros servicios
- **Escala independientemente:** Si hay pico de notificaciones, se escala sin afectar otros servicios
- **Integración externa:** Conecta con SendGrid (email), Firebase (push), Twilio (SMS)
- **Garantía de entrega:** Message broker reintenta si falla

2.3 Tecnología de Comunicación entre Microservicios

REST (HTTP/HTTPS) - Comunicación Sincrónica

Protocolos Seleccionados:

- **REST sobre HTTP/HTTPS** para consultas síncronas entre servicios
- **JSON** como formato de serialización de datos
- **JWT Bearer tokens** para autenticación

Justificación:

- ☒ **Eficiencia en transferencia de datos:** JSON es ligero y ampliamente soportado
- ☒ **Amplio soporte en herramientas y frameworks:** Disponible en todas las plataformas (Java, Python, Node.js, C#, etc.)
- ☒ **Facilidad de debugging y monitoreo:** URLs y payloads legibles, herramientas de monitoreo estándar
- ☒ **Escalabilidad horizontal:** Stateless, fácil de replicar en múltiples instancias
- ☒ **Compatibilidad con múltiples clientes:** Web, mobile, terceros, webhooks
- ☒ **API Gateway:** Un único punto de entrada centraliza autenticación y ruteo

Ejemplos de Comunicación Sincrónica:

- msCurso consulta disponibilidad en msInscripcion
- msPago valida inscripción antes de procesar pago
- msEvaluacion verifica estudiante existe en msUsuario

Message Broker (Kafka/RabbitMQ) - Comunicación Asincrónica

Protocolos Seleccionados:

- **Message Broker** (Kafka o RabbitMQ) para eventos desacoplados
- **Publicador-Suscriptor** (Pub/Sub pattern)
- **Garantía de entrega:** At-least-once semantics
- **Dead Letter Queue (DLQ):** Para manejar eventos que fallan

Justificación:

- ☒ **Desacoplamiento total:** Servicios no necesitan conocerse directamente
- ☒ **Escalabilidad asincrónica:** Procesamiento de eventos en background sin bloquear
- ☒ **No satura el sistema:** msNotificacion procesa a su propio ritmo, sin afectar otros servicios
- ☒ **Confiabilidad:** Los mensajes se almacenan hasta ser procesados
- ☒ **Resiliencia:** Si un servicio falla, los eventos se reintentarán automáticamente
- ☒ **Auditoría:** Histórico completo de eventos para debugging
- ☒ **Flexibilidad de canales:** msNotificacion puede enviar por email, push, SMS sin afectar la operación original

Por qué msNotificacion DEBE ser asincrónico:

- Las operaciones de usuario no pueden esperar a que se envíe un email/push (potencialmente 1-5 segundos)
- Si un servicio de email cae, no debe afectar inscripciones, pagos o evaluaciones
- Picos de notificaciones (ej: calificaciones masivas) no pueden saturar el sistema
- Reintentos automáticos: Si SendGrid falla, el mensaje se reintenta sin intervención

Por qué msTarea usa comunicación HÍBRIDA (REST + Async):

- **Sincrónico:** Estudiante DEBE saber inmediatamente si su entrega fue aceptada (plazo, formato, inscripción)
- **Asincrónico:** Análisis pesado (plagio, antivirus) no puede bloquear la respuesta REST

- **Escalabilidad:** Durante picos de entrega, escalar workers de análisis sin afectar aceptación de entregas
- **UX:** Confirmación inmediata (50-100ms) pero análisis en background (1-10 segundos)

Por qué msPago usa Webhooks (asincronía):

- Los bancos tardan 1-5 segundos en confirmar transacciones
- El usuario no debe esperar confirmación del banco para recibir respuesta
- Los webhooks permiten confirmaciones asincrónicas sin polling
- Permite reintentos automáticos si la confirmación falla

Eventos Asincronos en el Message Broker

Todos los eventos siguen el patrón **Pub/Sub** con garantía **At-least-once**:

Evento	Publicador	Suscriptor(es)	Descripción	Ejemplo Payload
usuario.registrado	msUsuario	msNotificacion	Nuevo usuario creado	<code>{"idUsuario":"U1","email":"juan@ejemplo.com","nombre":"Juan"}</code>
inscripcion.creada	msInscripcion	msPago, msNotificacion	Estudiante se inscribe	<code>{"idInscripcion":"I1","idEstudiante":"U1","idCurso":"C1"}</code>
pago.iniciado	msPago	msNotificacion	Pago iniciado (espera confirmación)	<code>{"idPago":"P1","monto":120000,"estado":"PENDIENTE_CONFIRMACION"}</code>
pago.aprobado ★	msPago (webhook del banco)	msInscripcion, msNotificacion	Banco confirma pago	<code>{"idPago":"P1","idInscripcion":"I1","referenciaBanco":"TXN123456"}</code>
pago.rechazado	msPago (webhook del banco)	msNotificacion	Banco rechaza transacción	<code>{"idPago":"P1","motivo":"Fondos insuficientes"}</code>
tarea.creada	msTarea	msNotificacion	Profesor crea tarea	<code>{"idTarea":"T1","idCurso":"C1","titulo":"Actividad 1"}</code>
tarea.entregada ★	msTarea (REST validado)	Auditoría, msTarea workers	Entrega aceptada (validación completada en REST)	<code>{"idEntrega":"E1","idTarea":"T1","idEstudiante":"U1","estado":"ENTREGADA"}</code>
tarea.procesada ★	msTarea (workers async)	msNotificacion, msEvaluacion	Análisis completado (plagio, antivirus)	<code>{"idEntrega":"E1","estado":"COMPLETADA","plagio":0.12,"antivirus":"LIMP"}</code>
evaluacion.creada	msEvaluacion	msNotificacion	Profesor crea evaluación	<code>{"idEvaluacion":"EV1","titulo":"Prueba 1","ponderacion":30}</code>
evaluacion.calificada	msEvaluacion	msNotificacion	Profesor califica al estudiante	<code>{"idEvaluacion":"EV1","idEstudiante":"U1","nota":6.5,"feedback":"Buen trabajo"}</code>
curso.creado	msCurso	msNotificacion	Académico crea curso	<code>{"idCurso":"C1","nombre":"Backend Spring Boot"}</code>
inscripcion.cancelada	msInscripcion	msPago, msNotificacion	Estudiante cancela inscripción	<code>{"idInscripcion":"I1","motivo":"Cambio de decisión"}</code>

★ = Eventos críticos para asincronía (procesamiento pesado o larga duración)

Nota sobre tarea.entregada vs tarea.procesada:

- **tarea.entregada:** Publicado inmediatamente cuando REST valida (plazo, formato, inscripción) ✅ 50-100ms
- **tarea.procesada:** Publicado cuando análisis termina (plagio, antivirus) ⏳ 1-10 segundos
- Esto permite a msNotificacion alertar al profesor en dos momentos: entrega recibida (rápido) y análisis completo (detallado)

Ejemplo de Arquitectura Asincrónica:

```

Usuario se inscribe
↓
msInscripcion crea inscripción en BD
↓

```

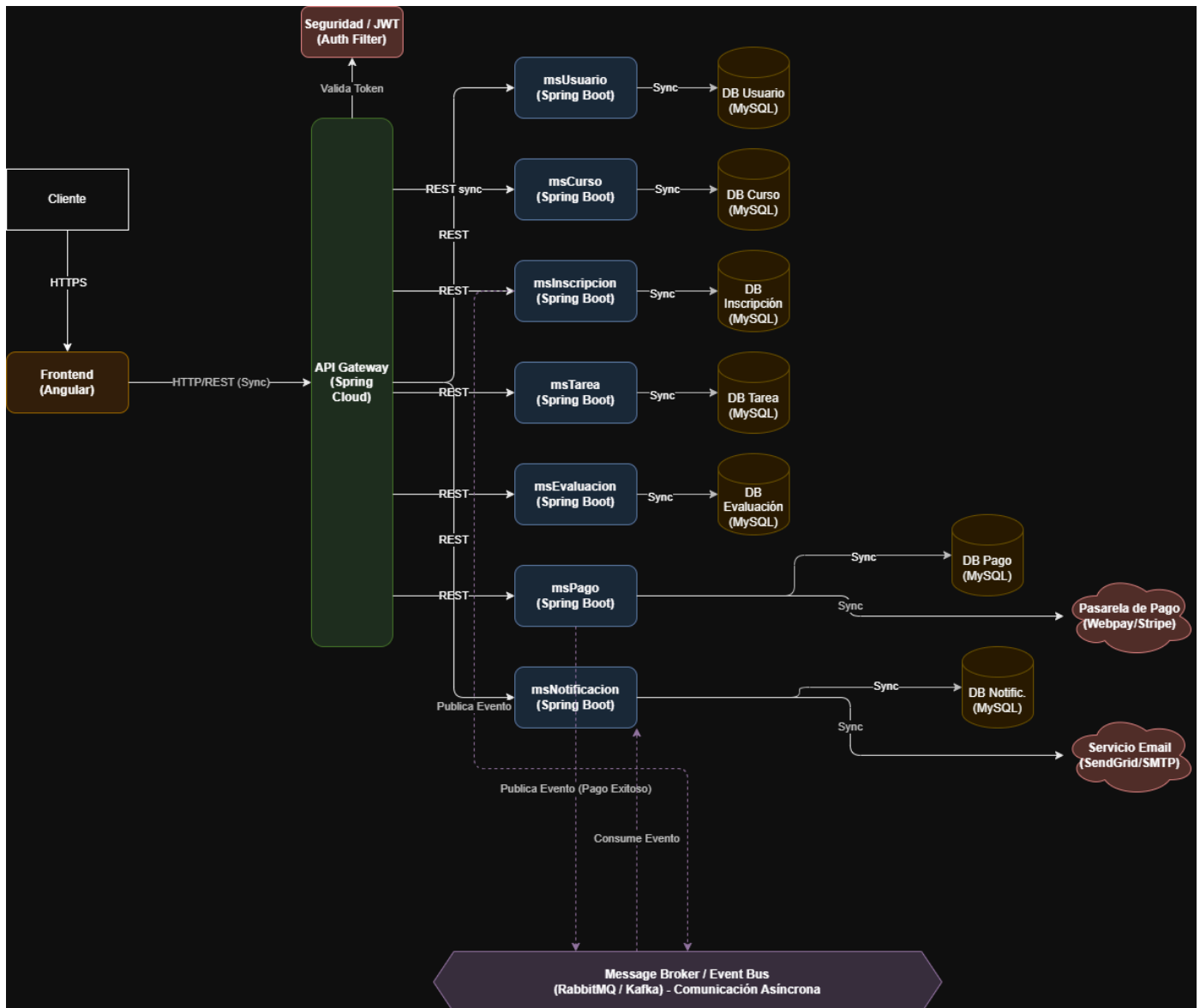
```

msInscripcion publica evento "inscripcion.creada"
↓
[RESPUESTA INMEDIATA AL USUARIO: 201 Created]
↓
Message Broker almacena el evento
↓
msNotificacion consume evento en background
↓
Envía email/push de confirmación (puede tomar 2-5 segundos)
↓
Registra histórico en MongoDB

```

⌚ TIEMPO TOTAL: Usuario recibe respuesta en 50ms + notificación en background en 2-5s
 ❌ SI FUERA SINCRÓNICO: Usuario esperaría 5+ segundos para respuesta

2.4 Arquitectura General



Descripción del diagrama:

La arquitectura implementa un patrón de microservicios con los siguientes componentes:

- **Cliente (Web/Mobile)**: Interfaz de usuario que inicia las solicitudes
- **API Gateway**: Punto de entrada único con enrutamiento, autenticación JWT y rate limiting
- **Microservicios (REST Sync)**: msUsuario, msCurso, msInscripcion, msTarea, msEvaluacion, msPago
 - Comunicación sincrónica para validaciones rápidas y consultas de datos
 - Latencia: 50-200ms
- **Message Broker**: Kafka/RabbitMQ para comunicación asíncrona
- **msNotificacion**: Consumidor asíncrono que procesa eventos en background
- **Servicios Externos**: SendGrid (Email), Firebase (Push), Twilio (SMS)

Flujos de Comunicación:

- 1. **Sincrónico (REST):** msUsuario ↔ msCurso ↔ msInscripcion ↔ msTarea ↔ msEvaluacion ↔ msPago
- 2. **Asincrónico (Events):** Todos los servicios → Message Broker → msNotificacion
 - Latencia aceptable: 1-5 segundos para notificación final
 - No bloquea operaciones del usuario

3. Especificación de Endpoints por Microservicio

 MICROSERVICIO: msUsuario

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
POST	Crear	Crear un nuevo usuario en la plataforma	/api/usuario/registro	Público	{ "email": "correo@ejemplo.com", "password": "pwd123", "nombre": "" }
POST	Crear	Autenticar usuario e iniciar sesión	/api/usuario/ingreso	Público	{ "email": "correo@ejemplo.com", "password": "pwd123" }
POST	Crear	Cerrar sesión del usuario autenticado	/api/usuario/salida	ESTUDIANTE, ACADEMICO, ADMIN	{ "token": "jwt-token" }
POST	Crear	Solicitar restablecimiento de contraseña	/api/usuario/restablecer-clave	Público	{ "email": "correo@ejemplo.com" }
PATCH	Actualizar	Restablecer contraseña mediante token de recuperación	/api/usuario/restablecer-clave/{token}	Público	{ "nuevaPassword": "NuevaPassword123" }
GET	Leer	Lista todos los usuarios registrados	/api/usuario	ACADEMICO, ADMIN	{ "token": "jwt-token" }
GET	Leer	Obtiene la información del usuario autenticado	/api/usuario/yo	ESTUDIANTE, ACADEMICO, ADMIN	{ "token": "jwt-token" }
GET	Leer	Obtiene la información de un usuario por ID	/api/usuario/{id}	ACADEMICO, ADMIN	{ "token": "jwt-token" }
PUT	Actualizar	Actualiza la información personal del usuario autenticado	/api/usuario/yo	ESTUDIANTE, ACADEMICO, ADMIN	{ "nombre": "Juan Carlos", "apellidoP": "Pérez", "apellidoM": "Gon" }
PUT	Actualizar	Actualiza la información de un usuario	/api/usuario/{id}	ADMIN	{ "nombre": "Pedro", "apellidoP": "Ramírez", "apellidoM": "Soto", "
PATCH	Actualizar	Actualiza el rol de un usuario	/api/usuario/{id}/rol	ADMIN	{ "nuevoRol": "ACADEMICO", "token": "jwt-token" }

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
PATCH	Actualizar	Actualiza la contraseña del usuario autenticado	/api/usuario/{id}/password	ESTUDIANTE, ACADEMICO, ADMIN	{ "oldPassword": "pwd123", "newPassword": "NuevaPassword123", "token": "jwt-token" }
PATCH	Actualizar	Bloquea o desbloquea un usuario	/api/usuario/{id}/estado	ADMIN	{ "estado": "BLOQUEADO", "motivo": "Incumplimiento de normas", "token": "jwt-token" }
DELETE	Borrar	Elimina un usuario de la plataforma	/api/usuario/{id}	ADMIN	{ "token": "jwt-token" }

 MICROSERVICIO: msCurso

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
POST	Crear	Crear un nuevo curso	/api/curso	ACADEMICO	{ "nombre": "Backend Spring Boot", "descripcion": "Curso de microservicios", "categoria": "Programacion", "precio": 120000, "fechaInicio": "2026-06-01", "fechaFin": "2026-08-01", "token": "jwt-token" }
GET	Leer	Lista todos los cursos disponibles	/api/curso	Público	{}
GET	Leer	Obtiene el detalle de un curso	/api/curso/{id}	Público	{}
PUT	Actualizar	Actualiza la información de un curso	/api/curso/{id}	ACADEMICO	{ "nombre": "Backend Spring Avanzado", "precio": 150000, "cupos": 40, "token": "jwt-token" }
PATCH	Actualizar	Cambia el estado del curso	/api/curso/{id}/estado	ACADEMICO	{ "estado": "INACTIVO", "token": "jwt-token" }
DELETE	Borrar	Elimina un curso	/api/curso/{id}	ADMIN	{ "token": "jwt-token" }

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
GET	Leer	Lista los cursos creados por un profesor	/api/curso/profesor/{id}	ACADEMICO, ADMIN	{"token":"jwt-token"}
GET	Leer	Busca cursos por categoría	/api/curso/categoria/{nombre}	Público	{}

 MICROSERVICIO: msInscripcion

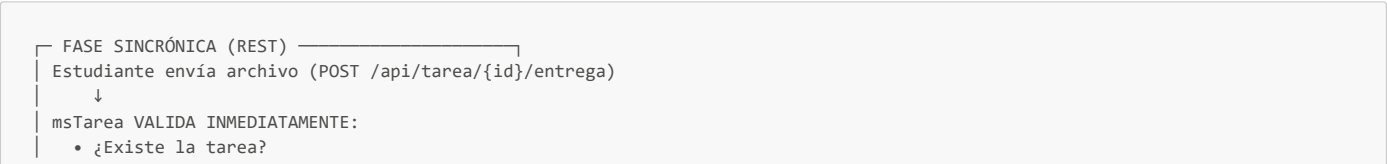
Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)	Respuesta (JSON)
POST	Crear	Inscribir un estudiante en un curso	/api/inscripcion	ESTUDIANTE	{"idCurso":"C1","token":"jwt-token"}	{"mensaje":"Inscripción realizada","idInscripcion"}
GET	Leer	Lista las inscripciones del estudiante autenticado	/api/inscripcion/yo	ESTUDIANTE	{"token":"jwt-token"}	[{"idInscripcion":"I1","cuBoot","estado":"ACTIVA"}]
GET	Leer	Lista todos los estudiantes inscritos en un curso	/api/inscripcion/curso/{id}	ACADEMICO, ADMIN	{"token":"jwt-token"}	[{"idUsuario":"U1","nombre
PATCH	Actualizar	Cambia el estado de la inscripción	/api/inscripcion/{id}/estado	ADMIN	{"estado":"ACTIVA","token":"jwt-token"}	{"mensaje":"Estado de insc
DELETE	Borrar	Cancela una inscripción	/api/inscripcion/{id}	ESTUDIANTE, ADMIN	{"token":"jwt-token"}	{"mensaje":"Inscripción ca

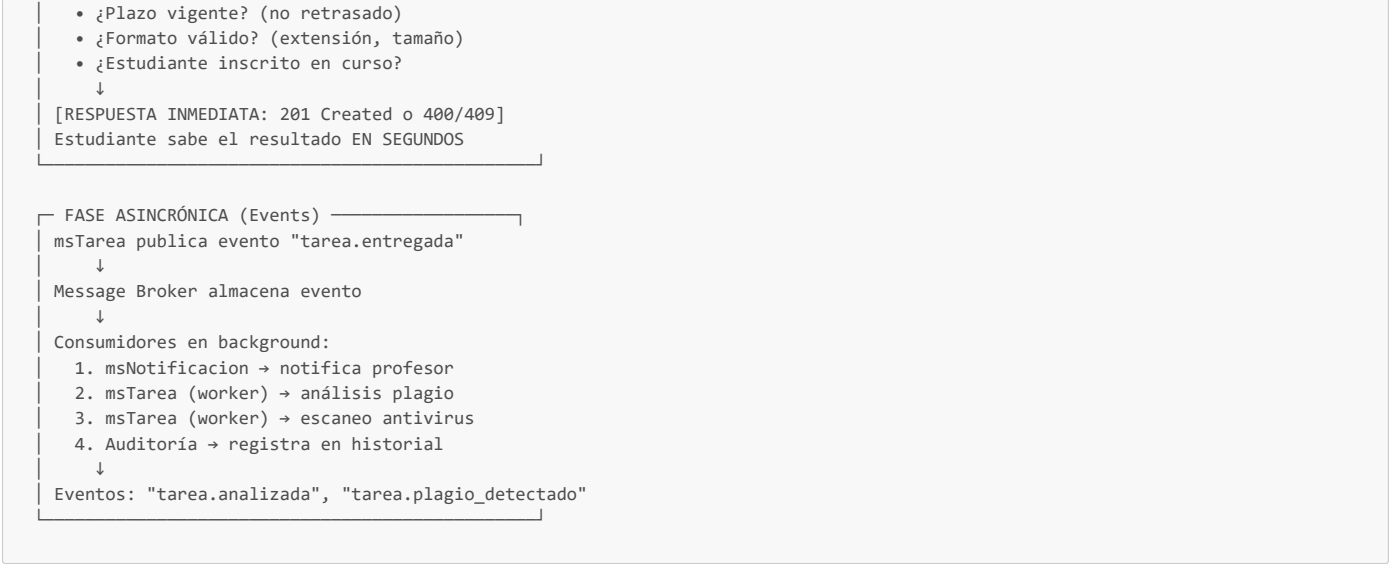
 MICROSERVICIO: msTarea (Híbrido: Sincrónico + Asincrónico)

Nota Arquitectónica Importante:

La entrega de tareas utiliza comunicación **síncrona REST** debido a que el estudiante requiere confirmación inmediata del resultado de la operación. Sin embargo, los procesos secundarios derivados de la entrega, como notificaciones, auditoría y habilitación de evaluación, se realizan mediante comunicación **asincrónica basada en eventos** para desacoplar los microservicios y mejorar la escalabilidad del sistema.

Flujo Híbrido REST + Async:





Estados de Entrega:

- **ENTREGADA** (síncrono): Archivo recibido, validación pasó
- **PROCESANDO** (asíncrono): Análisis de plagio/antivirus en curso
- **COMPLETADA** (asíncrono): Análisis finalizado, disponible para profesor
- **RECHAZADA** (síncrono o asíncrono): Formato inválido, plagio detectado, o malware

Beneficios del Enfoque Híbrido:

- **UX Rápida:** Estudiante sabe inmediatamente si su entrega fue aceptada (50-100ms)
- **Escalabilidad Asíncrona:** Análisis pesado no bloquea otras APIs
- **Validación Crítica:** Plazo, formato, inscripción validados al instante
- **Desacoplamiento:** Notificaciones y análisis independientes de la aceptación

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
POST	Crear	Crear una tarea para un curso	/api/tarea	ACADEMICO	{ "idCurso": "C1", "titulo": "Actividad 1", "descripcion": "Crear API REST", "fechaEntrega": "2026-06-10", "token": "jwt-token" }
GET	Leer	Lista tareas de un curso	/api/tarea/curso/{id}	ESTUDIANTE, ACADEMICO	{ "token": "jwt-token" }
POST	Crear	Entrega de tarea por estudiante (SÍNCRONO)	/api/tarea/{id}/entrega	ESTUDIANTE	{ "archivo": "base64file", "comentario": "Adjunto tarea", "token": "jwt-token" }
PUT	Actualizar	Reemplaza entrega de tarea (SÍNCRONO)	/api/tarea/{id}/entrega/{idEntrega}	ESTUDIANTE	{ "archivo": "base64file_nuevo", "token": "jwt-token" }
GET	Leer	Lista entregas de una tarea	/api/tarea/{id}/entregas	ACADEMICO	{ "token": "jwt-token" }

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
GET	Leer	Obtener estado/análisis de entrega	/api/tarea/entrega/{id}	ESTUDIANTE, ACADEMICO	{"token":"jwt-token"}
PUT	Actualizar	Actualiza una tarea	/api/tarea/{id}	ACADEMICO	{"titulo":"Actividad REST", "fechaEntrega":"2026-06-15", "token":"jwt-token"}
DELETE	Borrar	Elimina una tarea	/api/tarea/{id}	ACADEMICO	{"token":"jwt-token"}
(Event)	-	Evento: Entrega procesada	-	Sistema	{"idEntrega":"E1", "plagio":0.12, "antivirus":"LIMPIO"}

MICROSERVICIO: msEvaluacion

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
POST	Crear	Crear evaluación para un curso	/api/evaluacion	ACADEMICO	{"idCurso":"C1", "titulo":"Prueba 1", "ponderacion":30, "fecha":"2026-06-20", "token":"jwt-token"}
GET	Leer	Lista evaluaciones de un curso	/api/evaluacion/curso/{id}	ESTUDIANTE, ACADEMICO	{"token":"jwt-token"}
POST	Crear	Registrar calificación de estudiante	/api/evaluacion/{id}/calificar	ACADEMICO	{"idEstudiante":"U1", "nota":6.5, "feedback":"Buen trabajo", "token":"jwt-token"}
GET	Leer	Ver notas del estudiante autenticado	/api/evaluacion/notas/yo	ESTUDIANTE	{"token":"jwt-token"}
PUT	Actualizar	Modificar una calificación	/api/evaluacion/{id}/calificacion/{idNota}	ACADEMICO	{"nota":5.9, "feedback":"Corregido", "token":"jwt token"}
DELETE	Borrar	Eliminar una evaluación	/api/evaluacion/{id}	ACADEMICO, ADMIN	{"token":"jwt-token"}

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)

 MICROSERVICIO: msPago

Nota Importante - Arquitectura Híbrida (Síncronica + Asíncronica):

msPago combina dos paradigmas de comunicación:

- **Síncronico (REST):** El usuario inicia el pago y recibe respuesta inmediata (buena UX)
- **Asíncronico (Webhooks):** Los bancos confirman pagos a través de webhooks, publicando eventos al Message Broker
 - El banco responde con `/webhook/pago/confirmacion` (callback)
 - msPago publica evento `pago.aprobado` o `pago.rechazado`
 - msInscripcion consume el evento para activar/cancelar la inscripción
 - msNotificacion consume el evento para enviar recibo/notificación

Flujo de Pago Híbrido:



Beneficios:

-  UX: Usuario recibe respuesta inmediata (50-100ms)
-  Confiabilidad: No depende del tiempo de respuesta del banco
-  Escalabilidad: Confirmaciones de bancos procesadas en paralelo
-  Resiliencia: Si banco cae, el evento se reintenta automáticamente

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
POST	Crear	Procesar pago de inscripción	<code>/api/pago</code>	ESTUDIANTE	<code>{"idInscripcion":"I1","metodoPago":"TARJETA","monto":12000,"token"}</code>
GET	Leer	Obtener detalle de un pago	<code>/api/pago/{id}</code>	ESTUDIANTE, ADMIN	<code>{"token":"jwt-token"}</code>
GET	Leer	Lista pagos del usuario autenticado	<code>/api/pago/yo</code>	ESTUDIANTE	<code>{"token":"jwt-token"}</code>
PATCH	Actualizar	Solicitar devolución de pago	<code>/api/pago/{id}/reembolso</code>	ESTUDIANTE, ADMIN	<code>{"motivo":"Cancelación curso","token":"jwt-token"}</code>

Verbo	CRUD	Detalle	Endpoint	Roles	Requisito (JSON)
GET	Leer	Lista todos los pagos	/api/pago	ADMIN	{"token":"jwt-token"}
(Webhook)	Actualizar	Confirmación de pago desde banco	/webhook/pago/confirmacion	Sistema/Banco	{"idPago":"P1","estado":"APROBADO","referenciaBanco":"TXN1:"}

🔔 MICROSERVICIO: msNotificacion (Asincrónico)

Nota: msNotificacion funciona principalmente como **consumidor de eventos asincronos** del Message Broker. Los endpoints REST son opcionales (solo para consulta). Las notificaciones se envían a través de canales externos (email, push, SMS) y **NO son sincronos**.

Verbo	CRUD	Detalle	Endpoint	Roles	Descripción	Comportamiento
(Async)	Crear	Evento: Usuario registrado	-	Sistema	Consumidor del evento <code>usuario.registrado</code>	Envía email de bienvenida vía SendGrid
(Async)	Crear	Evento: Inscripción creada	-	Sistema	Consumidor del evento <code>inscripcion.creada</code>	Envía confirmación de inscripción por email/push
(Async)	Crear	Evento: Pago aprobado	-	Sistema	Consumidor del evento <code>pago.aprobado</code>	Envía recibo de pago por email
(Async)	Crear	Evento: Tarea entregada	-	Sistema	Consumidor del evento <code>tarea.entregada</code>	Envía confirmación al profesor por push
(Async)	Crear	Evento: Evaluación calificada	-	Sistema	Consumidor del evento <code>evaluacion.calificada</code>	Envía resultado de evaluación al estudiante
(Async)	Crear	Evento: Tarea próxima a vencer	-	Sistema	Consumidor de eventos <code>scheduled</code>	Envía recordatorio 24 horas antes
GET	Leer	Obtener historial de notificaciones	/api/notificacion/yo	ESTUDIANTE, ACADEMICO, ADMIN	Consulta notificaciones recibidas	Sincrónico: Devuelve histórico almacenado en BD
PATCH	Actualizar	Marcar notificación como leída	/api/notificacion/{id}/leida	ESTUDIANTE, ACADEMICO, ADMIN	Actualiza estado de notificación	Sincrónico: Marca como leída en MongoDB
DELETE	Borrar	Eliminar notificación	/api/notificacion/{id}	ESTUDIANTE, ADMIN	Borra notificación del histórico	Sincrónico: Soft delete de notificación

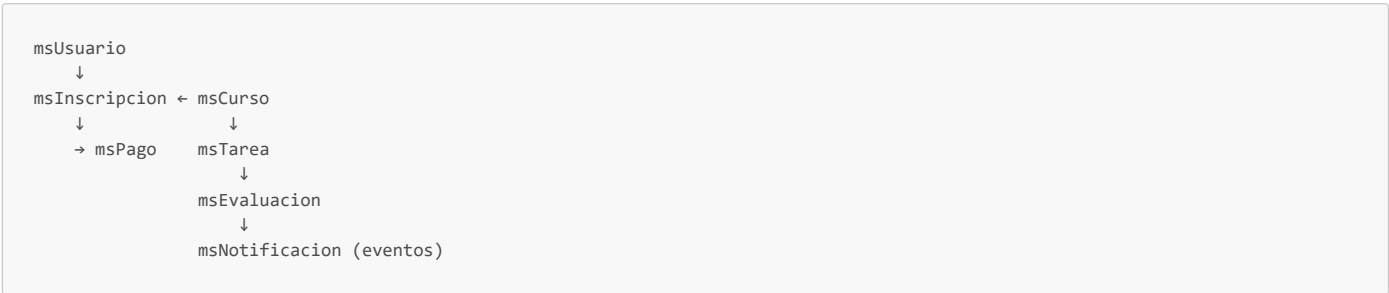
Canales de Entrega:

- 📧 **Email:** SendGrid (máx 100,000 emails/mes en plan gratuito)
- 🔔 **Push:** Firebase Cloud Messaging (FCM)
- 📱 **SMS:** Twilio (solo para notificaciones críticas)

Garantías:

- ✅ At-least-once delivery: Reintenta hasta 3 veces si falla
- ✅ Dead Letter Queue: Eventos fallidos se guardan para análisis
- ✅ No bloquea operaciones: Otras apis responden inmediatamente

5. Matriz de Comunicación entre Microservicios



Flujos de Comunicación Principales

1. **Registro de Usuario:** Usuario → Crear en msUsuario → token JWT
2. **Inscripción a Curso:** Estudiante → msInscripcion → verifica Curso (msCurso) → estado PENDIENTE_PAGO
3. **Pago (Híbrido):**
 - Estudiante → msPago (REST) → devuelve 201 inmediato
 - Banco → Webhook → msPago → publica `pago.aprobado` (async)
 - msInscripcion consume evento → actualiza ACTIVA
4. **Asignación de Tareas:** Profesor → msTarea → asocia Curso (msCurso)
5. **Entrega de Tarea (Híbrido REST + Async):**
 - Estudiante → msTarea (REST sincrónico) → valida plazo, formato, inscripción
 - Devuelve 201 Created con estado `ENTREGADA` (confirmación inmediata)
 - msTarea publica evento `tarea.entregada` (async) → inicia análisis
 - msTarea publica `tarea.procesada` cuando análisis finaliza → msNotificacion notifica profesor
6. **Calificación:** Profesor → msEvaluacion → genera evento → msNotificacion (envía notificación)
7. **Notificaciones Automáticas:** Todos los eventos → Message Broker → msNotificacion consume y envía

6. Escalabilidad de la Arquitectura

5.1 Escalabilidad Horizontal Independiente

La arquitectura soporta el crecimiento mediante escalabilidad horizontal independiente de cada microservicio, sin afectar a otros:

Ejemplo 1: Pico de Inscripciones en Semestre

Escenario: Durante el periodo de "Inscripción de Semestre", el tráfico hacia el catálogo de cursos se dispara exponencialmente.

Solución:

- El sistema puede levantar **10 nuevas instancias de msCurso** sin malgastar recursos escalando msUsuario
- Load Balancer distribuye las 10 instancias de msCurso
- msUsuario mantiene sus 2-3 instancias normales (autenticación no es el cuello de botella)
- msPago también escala independientemente si es necesario
- **Resultado:** Catálogo fluido, sin degradación en otros servicios

Antes:	Después del escalado:
msCurso: 1 →	msCurso: 10 ✓
msUsuario: 1	msUsuario: 1 (sin cambios)
msPago: 1	msPago: 1-2 (escalado si es necesario)

Ejemplo 2: Pico de Carga en Evaluaciones

Escenario: En época de exámenes finales, se produce subida masiva de archivos de entregas, aumento de calificaciones y notificaciones.

Solución:

- **Escalar msEvaluacion** a 8+ instancias para procesar calificaciones rápidamente
- **Escalar msTarea** a 6+ instancias para almacenar archivos de entregas
- **Escalar msNotificacion** a 5+ instancias para enviar notificaciones sin delay
- Los demás servicios (msCurso, msUsuario, msPago) no necesitan escalarse
- **Resultado:** Rendimiento fluido durante exámenes, sin recursos ociosos en otros servicios

Época Normal:	Época de Exámenes:
msTarea: 1	msTarea: 6+ ✓
msEvaluacion: 1 →	msEvaluacion: 8+ ✓
msNotificacion: 1	msNotificacion: 5+ ✓
msCurso: 1	msCurso: 1 (sin cambios)

5.2 Beneficios de la Escalabilidad

- **Optimización de Costos:** Escalar solo los servicios necesarios, ahorrando en infraestructura
- **Alta Disponibilidad:** Si una instancia falla, otras toman el tráfico automáticamente
- **Mejora de Performance:** Reducción de latencia en operaciones críticas
- **Independencia Tecnológica:** Cada servicio puede usar su propia tecnología óptima
- **Escalabilidad Asincrónica de Notificaciones:** msNotificacion procesa eventos a su propio ritmo sin afectar otros servicios

Ejemplo 3: Masivo de Notificaciones

Escenario: El profesor carga calificaciones finales para 500 estudiantes, generando 500 eventos de `evaluacion.calificada`.

Sin Asincronía (MAL):

- Si msPago fuera sincrónico y esperara respuesta de notificación, cada calificación tomaría 5+ segundos

- Total: $500 \times 5s = 2500$ segundos (41 minutos) 🕒
- Sistema se congela

Con Asincronía (BIEN):

- msPago publica evento en 50ms
- Devuelve respuesta inmediata al usuario
- Message Broker almacena los 500 eventos
- msNotificacion los procesa en paralelo (8+ workers)
- Total: ~60 segundos de procesamiento en background
- Usuario no espera, se reciben notificaciones gradualmente

```
Timeline:
T=0ms:      Profesor carga 500 calificaciones
T=50ms:     msPago devuelve 200 OK (sin esperar notificaciones)
T=50-2000ms: msNotificacion procesa eventos en paralelo
T=2000ms:   Últimas notificaciones enviadas
```

7. Seguridad y Protección de Datos

6.1 Autenticación

JWT (JSON Web Token)

- **Emisión:** Al hacer login en msUsuario, se genera un JWT válido por 24 horas
- **Transmisión:** Token incluido en header `Authorization: Bearer {jwt-token}`
- **Validación:** Cada microservicio valida el JWT antes de procesar solicitudes
- **Claims JWT:** Incluye id usuario, rol, email, timestamp de expiración
- **Secreto compartido:** Todos los servicios comparten la misma clave privada para validar

```
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "U1",
    "email": "usuario@ejemplo.com",
    "rol": "ESTUDIANTE",
    "iat": 1620000000,
    "exp": 1620086400
  }
}
```

6.2 Control de Acceso (RBAC - Role-Based Access Control)

Roles Definidos:

- **Público:** Acceso sin autenticación (registro, login, listar cursos)
- **ESTUDIANTE:** Acceso a cursos inscritos, tareas, pagos propios, calificaciones
- **ACADEMICO:** Creación de cursos, tareas, evaluaciones; calificación
- **ADMIN:** Control total, gestión de usuarios, pagos, sistema

Validación por Endpoint:

```
✓ Permitido: GET /api/usuario/yo (token requerido, cualquier rol)
✓ Permitido: PATCH /api/usuario/{id}/rol (solo ADMIN)
✗ Rechazado: POST /api/pago (solo ESTUDIANTE, no ACADEMICO)
✗ Rechazado: DELETE /api/curso/{id} (solo ADMIN, no ACADEMICO)
```

Implementación:

- Middleware de autorización en API Gateway valida rol del token
- Cada servicio verifica nuevamente el rol en endpoints sensibles
- Logs de acceso denegado para auditoría

6.3 Encriptación de Datos

En Tránsito (En Tránsito):

- **HTTPS/TLS 1.2+:** Todos los endpoints utilizan HTTPS

- **Certificados SSL válidos:** Emitidos por autoridades certificadoras confiables
- **API Gateway força HTTPS:** Rechaza conexiones HTTP sin cifrar

En Reposo (Base de Datos):

- **Contraseñas:** Hasheadas con bcrypt (salt rounds ≥ 10)
- **Tokens de recuperación:** Hasheados y con fecha de expiración
- **Datos sensibles:** Encriptación a nivel de base de datos (transparent data encryption)
- **Información de pago:** PCI-DSS compliance (no almacenar números de tarjeta, solo tokens)

Ejemplo de Hash de Contraseña:

```
Password: "MySecure123Pass"
Salt: [generado aleatoriamente]
Hash: $2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcg7b3XeKeUxWdeS86E36P4/liG
```

6.4 Normativas y Cumplimiento

Normativa GDPR (Protección de Datos Personales):

- ☒ Derecho al olvido: Endpoint de eliminación de usuario elimina todos sus datos
- ☒ Portabilidad: Acceso a datos personales en formato JSON
- ☒ Consentimiento: Requerido en registro para términos de servicio
- ☒ Privacidad: Datos no compartidos sin consentimiento explícito

Normativa PCI-DSS (Pagos):

- ☒ No almacenar números de tarjeta completos
- ☒ Usar tokenización de terceros (Stripe, PayPal)
- ☒ Encriptación de datos de transacciones
- ☒ Auditoría de acceso a datos de pagos

Normativa LPD (Ley de Protección de Datos - Chile):

- ☒ Consentimiento informado para recolección de datos
- ☒ Acceso a datos personales sin costo
- ☒ Rectificación/eliminación de datos incorrectos
- ☒ Restricción en transferencia internacional de datos

6.5 Prácticas de Seguridad

Validación de Input:

- Email válido (RFC 5322)
- Contraseña: Mínimo 8 caracteres, mayúscula, número, símbolo
- Sanitización de strings (prevenir SQL injection)
- Validación de tipos de datos JSON

Rate Limiting:

- Máximo 5 intentos de login por IP en 15 minutos
- Máximo 100 requests por minuto por usuario autenticado
- Protección contra fuerza bruta

Auditoría y Logging:

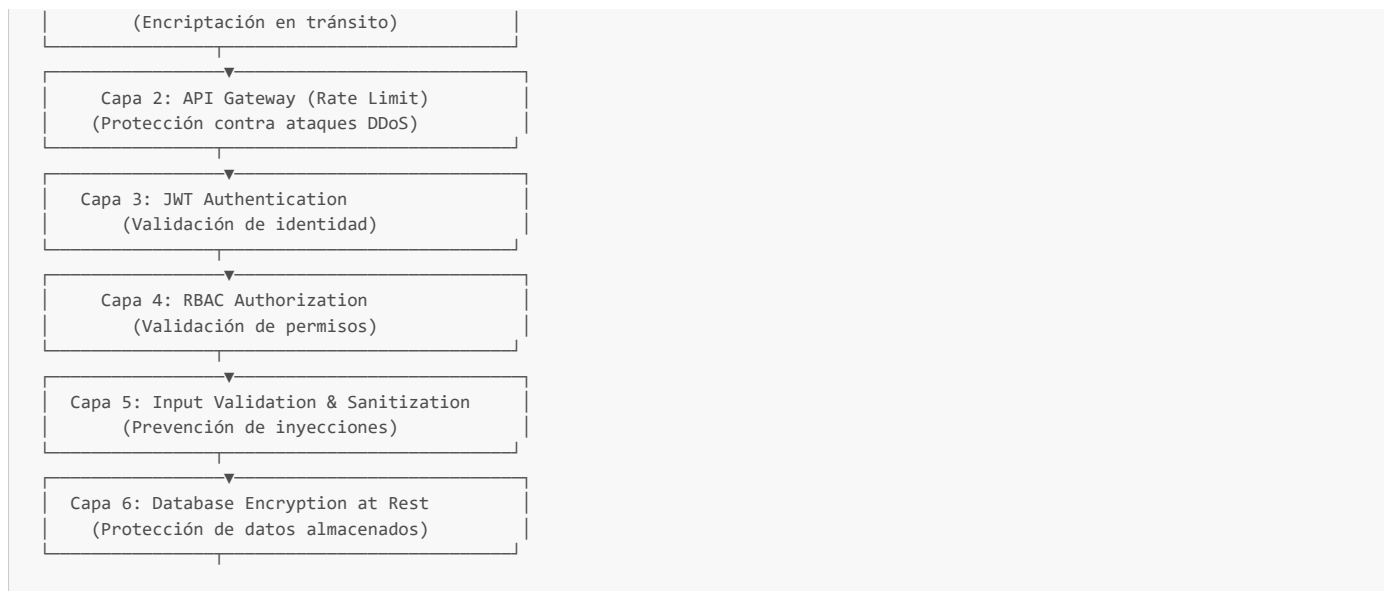
- Registro de todos los accesos (login, logout, cambio de contraseña)
- Registro de acceso denegado (intentos de acceso no autorizado)
- Logs almacenados por mínimo 90 días
- Análisis de anomalías automático

Gestión de Secretos:

- JWT secret almacenado en variables de entorno
- API keys de terceros en secrets manager (HashiCorp Vault, AWS Secrets Manager)
- Rotación de claves cada 90 días
- Nunca hardcodear secretos en código

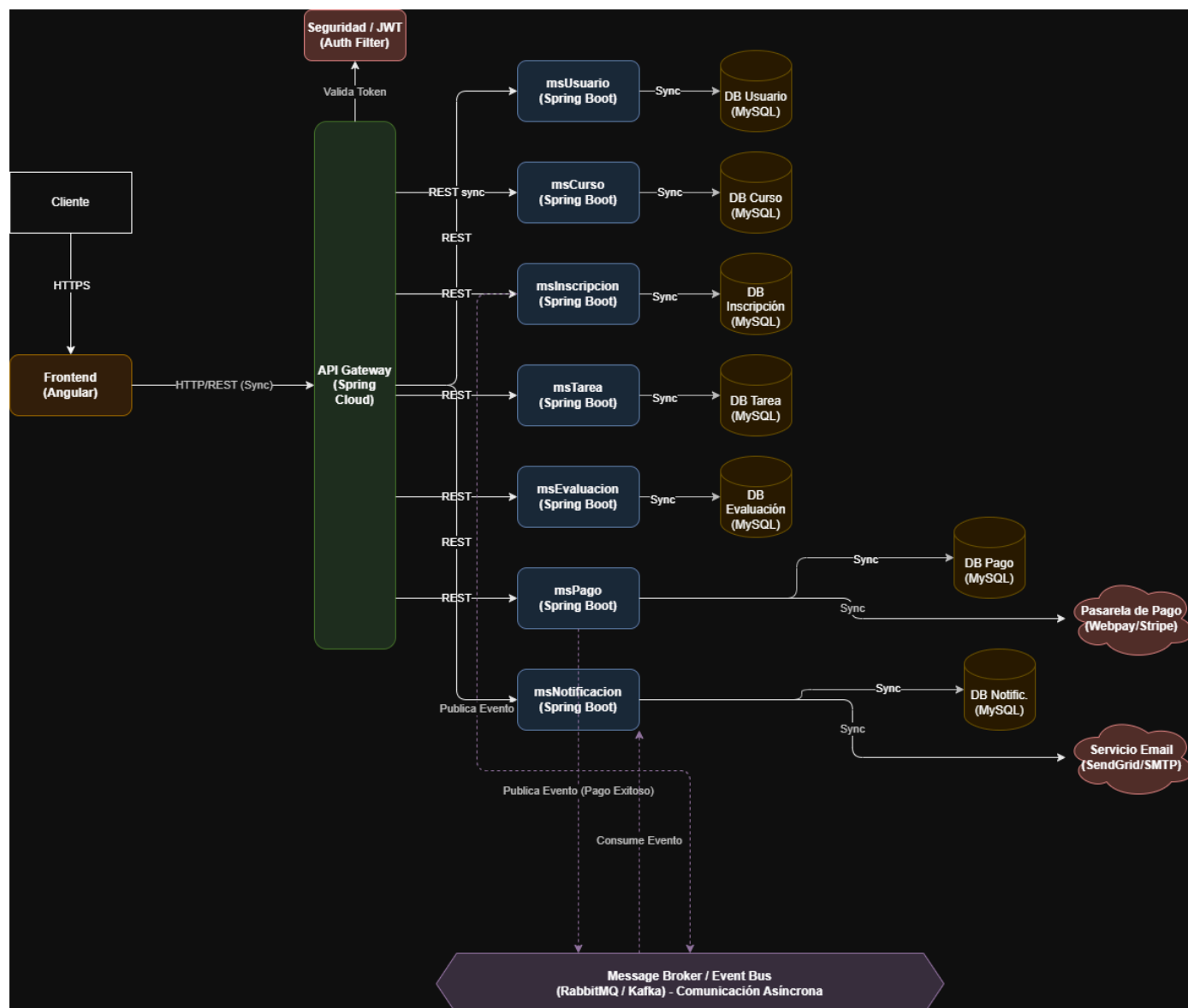
8. Patrones de Seguridad Implementados

Defensa en Profundidad



9. Diagrama de Arquitectura de Microservicios

La siguiente ilustración presenta la arquitectura global de la plataforma de aprendizaje, mostrando cómo se comunican los microservicios a través de REST (comunicación sincrónica) y un Message Broker (comunicación asincrónica):



Descripción del diagrama:

- Cliente (Web/Mobile):** Interfaz de usuario

- **API Gateway:** Punto de entrada único con autenticación JWT y rate limiting
- **Microservicios (REST Sync):** msUsuario, msCurso, msInscripcion, msTarea, msEvaluacion, msPago
- **Bases de Datos:** Cada microservicio tiene su propia base de datos (MySQL)
- **Message Broker:** Kafka/RabbitMQ para comunicación asincrónica
- **msNotificacion:** Consume eventos y envía notificaciones a través de múltiples canales (Email, Push, SMS)
- **Servicios Externos:** SendGrid (Email), Firebase (Push), Twilio (SMS)

10. Estado y Avances: Proyecto Backend I - EFT Semana 9

Fecha: 09/05/2026 - 16:15 horas

Resumen técnico del estado de acuerdo con la rúbrica, lo implementado y subido a la rama "branch-rai" en repositorio del proyecto.

10.1 Arquitectura y Configuración Base

Monolito Modular (Fase de Desarrollo):

- Proyecto separando las responsabilidades por paquetes: `controller`, `service`, `repository`, `dto`, `mapper`, `aop`, `exception`.
- Esto permite simular microservicios dentro de un solo proyecto para facilitar las pruebas.
- **Decisión de Desarrollo:** Actualmente, el código está escrito como un monolito que integra la lógica para **dos microservicios principales (Cursos y Usuarios)**, tal como lo pide el ejemplo de la guía. En el **despliegue final**, se separará físicamente creando un microservicio independiente por dominio.

Estrategia de Base de Datos Dual (H2 / MySQL):

- **Entorno de Desarrollo y Pruebas (EFT):** Todo el código actual (`pom.xml` y `application.properties`) está configurado para utilizar **H2 Database en memoria**. Esto facilita la revisión académica sin requerir levantar servicios externos y cumple estrictamente con las instrucciones de la evaluación.
- **Despliegue Final (Producción):** La arquitectura transicionará al uso de contenedores Docker con **MySQL** cuando los microservicios se separen.
- Se integra **HikariCP** para la gestión eficiente del pool de conexiones.
- **Spring Data JPA** genera y actualiza automáticamente las tablas de las entidades.
- Se cuenta con un script `init.sql` que puebla la base de datos de desarrollo con información de ejemplo para pruebas inmediatas.

Dependencias Configuradas:

- Spring Web (`spring-boot-starter-web`)
- Spring Data JPA (`spring-boot-starter-data-jpa`)
- H2 Database (`h2`) y soporte MySQL Driver (`mysql-connector-j` para futuro despliegue)
- Spring Boot Actuator (`spring-boot-starter-actuator`)
- Spring AOP (`spring-boot-starter-aop`)
- Validación de Datos (`spring-boot-starter-validation`)
- Pool de Conexiones HikariCP: Parametrizado explícitamente en `application.properties`
- Lombok para reducir boilerplate
- Jakarta Persistence API

Desarrollo de Servicios de Negocio:

- Implementación de al menos 3 servicios con buenas prácticas:
 - `UsuarioService`: CRUD completo de usuarios
 - `CursoService`: CRUD completo de cursos
 - `NotificacionService`: Gestión de notificaciones asincrónicas

CRUDs Completos Implementados:

- **Usuario y Curso:** Expuestos a través de `UsuarioController` y `CursoController`
- Las 4 operaciones (Create, Read, Update, Delete) completamente funcionales, documentadas y evidenciadas en Postman.
- Endpoints REST siguiendo convenciones RESTful.

10.2 Seguridad y Protección de Datos

Validación de Entrada:

- Implementada la librería de validación de Spring
- Los DTOs (`UsuarioRequestDTO`, `CursoRequestDTO`) usan anotaciones (`@NotBlank`, `@Email`, `@Size`, `@Min`).

Hashing de Contraseñas:

- Se implementa un workaround para el hashing en el escenario de esta EFT
- Se instancia `BCryptPasswordEncoder` de forma aislada, evitando la configuración completa de Spring Security
- Esto permite aplicar hashing a las contraseñas sin bloquear los endpoints para pruebas con Postman
- Representa un mínimo viable para la entrega, de acuerdo con lo solicitado

Ocultamiento de Credenciales:

- Los Mappers están configurados para que `UsuarioResponseDTO` no devuelva la contraseña en las respuestas JSON.

10.3 Manejo Global de Errores

Centralización de Errores:

- Se implementa `GlobalExceptionHandler` con la anotación `@RestControllerAdvice`
- Si un usuario envía datos inválidos o busca un ID que no existe, la API devuelve respuestas estructuradas (`ApiException` con códigos HTTP 400, 404, 500).

10.4 Simulación de Microservicios (Comunicación Asíncrona)

Patrón Publicador/Suscriptor:

- Para cumplir con la sugerencia de comunicación simple, se utiliza **Spring Events** (simulando RabbitMQ/Kafka).
- Se implementa `UsuarioRegistradoEvent` como herramienta nativa.
- El evento se dispara y `NotificacionService` lo escucha en segundo plano (`@Async`), sin bloquear la respuesta al cliente.

10.5 Trazabilidad con Spring AOP

Aspect-Oriented Programming:

- Se implementa `GlobalAspect` que intercepta automáticamente todos los controladores para registrar el tiempo de petición mediante `Slf4j`.

10.6 Testing y Documentación

Colección Postman:

- Archivo `LearningPlatform.postman_collection.json` con 24 ejemplos de prueba (éxito y error).
- **Nota:** Se han incluido pruebas explícitas de la operación `DELETE` para Usuarios y Cursos con el fin de evidenciar el cumplimiento del Criterio 5.

Base de Datos de Prueba:

- Script `init.sql` auto-poblado con 4 usuarios de ejemplo y 3 cursos.

11. Conclusión

Esta especificación proporciona un contrato claro entre servicios, facilitando:

- **Desarrollo paralelo:** Múltiples equipos pueden trabajar en servicios independientes simultáneamente
- **Testing independiente:** Cada microservicio tiene sus propios tests unitarios e integración
- **Documentación clara:** Contratos JSON bien definidos para frontend y terceros
- **Escalabilidad:** Crecimiento horizontal independiente según demanda
- **Confiabilidad:** Aislamiento de fallos, recuperación automática, auditoría completa
- **Seguridad:** Autenticación JWT, autorización RBAC, encriptación, cumplimiento normativo
- **Mantenibilidad:** Responsabilidades claras, bajo acoplamiento, fácil actualización de servicios

Beneficios Clave de la Arquitectura

Beneficio	Implementación
Alta Disponibilidad	Múltiples instancias de cada servicio, API Gateway con health checks
Escalabilidad Elástica	Escalar servicios independientemente según carga
Resiliencia	Fallos aislados, circuit breakers, reintentos automáticos
Performance	APIs REST rápidas, cache con Redis, eventos asíncronos
Seguridad	JWT, RBAC, HTTPS, encriptación, auditoría
Flexibilidad	Tecnologías heterogéneas por servicio
Testabilidad	Servicios independientes, fácil de mockear
Deployabilidad	Deployment independiente, sin coordinación global